

3OpChangedNumber53 3OpRuleInversion51 3OpCrossrefConflict30 3OpDeletedCondition37 3Cat-
NumericMismatch50 3CatRuleInversion45 3CatCrossrefConflict39 3CatMissingCondition37 3CatD-
iff9 4WithRule98/98

SpecRead: A Benchmark for Measuring Whether Language Models Understand Hardware Specifications

Feilian Huang
Independent Researcher

September 2026

Abstract

Existing benchmarks for large language models (LLMs) in hardware design evaluate downstream artifacts such as generated RTL, assertions, or testbenches. When a model fails such a benchmark, the failure is ambiguous: it may have misread the specification, or it may have understood the specification and failed to write the code. We present SpecRead, a benchmark that isolates specification comprehension from generation ability. SpecRead v2.1 contains 385 questions over 10 open-source OpenTitan IP blocks, in four types: exact retrieval, cross-section reasoning, contradiction detection in mutated specifications, and spec-RTL consistency checking, plus 41 distractor controls and 41 consistent-RTL controls. Type-4 items are built from real RTL mutations; we retain only mutations that Icarus Verilog simulation shows to change observable behavior. A with-spec vs. without-spec ablation suggests the questions require the excerpt rather than training recall alone (without-spec accuracy 3/60), though it does not rule out memorization of the unmutated source text helping a model spot mutations. As an initial characterization with a small model, Ministral-3B scores 33.2% overall (128/385; macro average 39.0%) : 55.2% on retrieval, 51.7% on cross-section reasoning. On the two contradiction-focused types, the primary verdict-plus-location measure gives 48.0% (t3) and 63.3% (t4), with a 51.2% false-positive rate on self-consistent distractor passages and a 100% false-positive rate on consistent-RTL controls. Layered scoring shows the model locates contradictions well (78.9–81.6% location accuracy) but scores lower on their category (49.7–43.9% category accuracy), suggesting as a hypothesis that the fine-grained violation taxonomy is the hard part, though the category labels have not been independently validated and the finding is confounded with label ambiguity. A structured “rule-table” prompting intervention lowers accuracy on every question type except t2 (tied). SpecRead is automatically scorable by deterministic checks, with gray-zone cases counted wrong under the conservative main scoring; a deterministic lenient-threshold variant is reported as a sensitivity analysis in the appendix. The benchmark is regenerable for type-3 items via mutation injection, and built exclusively from public sources.

1 Introduction

Large language models are increasingly applied to hardware design: writing RTL, generating assertions, and summarizing specifications. The benchmarks used to evaluate them, including VerilogEval [1] and RTLLM [3], predominantly measure *generation*: given a specification or natural-language description, produce an artifact and check it by simulation or formal equivalence.

Generation benchmarks conflate two distinct capabilities. Consider a model that writes incorrect RTL for a UART baud-rate divider. Did it misread the specification (a comprehension failure), or did it understand the specification and fail at Verilog syntax, timing discipline, or state-machine

encoding (a production failure)? The benchmark cannot tell, and the distinction determines whether to invest in spec-grounded reasoning or in code synthesis.

SpecRead measures comprehension directly. Instead of asking the model to produce hardware, we ask it to *read*: retrieve facts from a specification, reason across sections, detect contradictions planted in mutated specifications, and judge whether an RTL excerpt is consistent with the specification it claims to implement. Every question is scored by deterministic checks, with gray-zone cases counted wrong under the conservative main scoring (see Scoring below). The focus is on types 3 and 4, which have no analogue in generation benchmarks:

- **Type 3 (contradiction detection).** We apply controlled mutations (number changes, rule inversions, cross-reference conflicts, deleted conditions) to real specification text and ask the model to locate and classify the contradiction. Distractor items, real unmutated passages that merely look suspicious, control the false-positive rate.
- **Type 4 (spec-RTL consistency).** We mutate real RTL, verify by simulation that the mutation changes observable behavior, and ask whether the mutated RTL still matches the specification. The model must connect natural-language rules to Verilog semantics.

Our contributions are: (1) the SpecRead benchmark (385 questions, 10 IPs, automatically scored); (2) a mutation-injection methodology that makes type-3 items regenerable and therefore more resistant to training-data contamination; (3) type-4 items filtered by simulation to guarantee an observable behavioral change; and (4) an initial single-model characterization on a small open model, with a fine-grained failure taxonomy. We frame the paper as a benchmark release plus this single-model characterization; with one 3B model evaluated, we make no claim that the difficulty profile is discriminative across models or that it transfers to larger or frontier models.

2 Related Work

RTL generation benchmarks. VerilogEval [1, 2] and RTLLM [3] established the paradigm of evaluating LLMs by the functional correctness of generated Verilog. Follow-up work extended it to assertions (AssertLLM [4], AssertLLM2 [5]), testbenches (AutoBench [6]), and multi-turn repair. These benchmarks measure the end of the pipeline; none isolates whether the model understood its input.

Contamination-resistant evaluation. Regenerable items, private held-out sets, and dynamic benchmark construction are active concerns across LLM evaluation [12, 13]. SpecRead’s mutation injection follows this spirit: because type-3 and type-4 items are generated by scripted mutation of public text, fresh instances can be produced on demand.

Indirect evaluations of comprehension. The closest existing work evaluates comprehension indirectly: generating RTL from a specification and then checking the RTL (round-trip), analyzing characteristic spec-misreading errors, or the reverse direction of RTL-to-spec summarization (RTL-BenchLS [7]). CVDP [8] includes an RTL-to/from-specification correspondence category (34 problems), but scores it with BLEU and LLM-based judging rather than deterministic gold labels. The nearest neighbors on each side are VClare [9], which injects specification defects including contradictions but measures repair outcomes (Verilog pass rate) rather than comprehension, and AssertLLM2 [5], whose bug-hunting setting uses mutated RTL but scores generated assertions rather than direct consistency judgments, without simulation-filtering mutations for observable difference.

To our knowledge, no existing benchmark directly scores specification comprehension itself: asking the model to detect, locate, and classify contradictions in a specification, or to judge spec-RTL consistency, against gold labels with scoring decoupled from generation ability. Characteristic spec-misreading failures (“Type II” errors) are a documented confound in RTL generation [10]; SpecRead isolates exactly that confound.

AI for EDA. Domain-adapted models such as ChipNeMo [11] and EDA copilots presuppose that models can read hardware documentation faithfully. SpecRead provides a direct measurement of that presupposition. Recent EDA question-answering work (Ask-EDA [21], ORD-QA [22], the EDA Corpus [23]) evaluates models on documentation-grounded QA but does not test contradiction detection or spec-RTL consistency.

Natural language inference and contradiction detection. Type-3 items are a domain-specific form of contradiction detection over long documents. General NLI benchmarks (DocNLI [17], ContractNLI [18]) and factual-consistency metrics (SummaC [19]) test whether models detect inconsistencies between texts, but on news or contract domains with different linguistic structure than hardware specifications. Long-document QA and contextual NLI (ConTRoL [20]) share the multi-hop reasoning demand of type-2 items. SpecRead differs in grounding contradictions in executable hardware semantics: type-4 mutations are simulation-verified to change observable behavior, a guarantee no text-only NLI benchmark provides.

3 Benchmark Design

3.1 Sources

All materials derive from the OpenTitan project [14] (Apache 2.0) at commit `80b5b321...` (full hash in the Reproducibility section). We record repository, file path, commit, and license for every excerpt. No proprietary or company-internal specifications are used.

3.2 Question types

Type 1: exact retrieval ($n=58$). Answer a factual question whose answer appears verbatim in the excerpt (e.g., a reset value or register address).

Type 2: cross-section reasoning ($n=58$). The answer requires combining statements from two or more sections (e.g., a programming sequence spanning the register list and the theory-of-operation section). Of the 58, 22 are multiple-choice with four options presented in the prompt (an earlier version omitted the option list on 15 of these; all were repaired in v2.1).

Type 3: mutated-spec contradiction detection ($n=171$). We apply one scripted mutation to a real excerpt and ask the model either to identify the contradiction (location and category) or to reply `NO_CONTRADICTION`. Mutation operators are *changed number* (e.g., “9/11/13 rounds” instead of the documented values), *rule inversion* (polarity flips), *cross-reference conflict* (a table row contradicting prose), and *deleted condition* (a dropped qualifier creating an over-generalization). The corresponding categories are `NUMERIC_MISMATCH`, `RULE_INVERSION`, `CROSSREF_CONFLICT`, and `MISSING_CONDITION`. Mutations are applied by exact-match scripted replacement, and the mutated sentence is kept fluent to avoid “looks edited” cues. Operator counts: `changed_number`

3OpChangedNumber, **rule_inversion** 3OpRuleInversion, **deleted_condition** 3OpDeletedCondition, **crossref_conflict** 3OpCrossrefConflict. The resulting gold categories are **NUMERIC_MISMATCH** 3CatNumericMismatch, **RULE_INVERSION** 3CatRuleInversion, **CROSSREF_CONFLICT** 3CatCrossrefConflict, **MISSING_CONDITION** 3CatMissingCondition (3CatDiff items’ gold category differs from the generating operator’s default mapping).

Type 4: spec-RTL consistency ($n=98$). We mutate real RTL (single-line substitutions or small deletions), simulate the original and mutated modules under Icarus Verilog [15] with a purpose-built testbench, and retain only mutations whose waveforms differ. Candidate RTL mutations were proposed by hand for each IP and simulated under purpose-built testbenches; only mutations with **RESULT: DIFFER** were retained, leaving 98. No single candidate/discard ledger was kept; per-IP review notes document 26 discarded candidates, dropped for behavioral equivalence under the testbench, Icarus Verilog quirks, or lack of a spec witness. A behavioral difference from the original RTL does not by itself imply a specification violation, since the changed behavior could fall in an unspecified region; at item-authoring time the author therefore wrote, for each of the 98 retained items, a gold answer citing the specific spec clause the mutation violates (a **rule** field plus an explanation linking the mutated line to that clause, present in 98/98 items). The model receives the specification excerpt and the mutated RTL excerpt and must judge consistency, citing the location and one of the four categories.

3.3 Distractor controls

The 41 distractor items are real, unmutated excerpts that look suspicious but are self-consistent: reset-value subtleties, level- vs. event-triggered interrupt semantics, reserved register fields, **x** reset values, and read-to-clear behavior. They are interleaved with the main set, so a model that flags anything unusual as a contradiction accumulates false positives. Of an initial 42 distractors, one (**uart-t3d-003**) was dropped as an exact duplicate prompt of **uart-t3d-001**. Distractors are presented in the type-3 format.

We also built 41 consistent-RTL controls for type 4: real, unmutated RTL excerpts paired with their specification passages, presented in the type-4 format, for which the correct verdict is **NO_CONTRADICTION**. Each control was independently audited for fairness (the excerpt must contain enough context to verify consistency); 24 excerpts were widened and 3 unfair controls replaced. These controls measure the false-positive rate on the violation-vs.-compliant decision specifically.

3.4 Scoring

The main results use conservative automatic scoring: deterministic checks only, with every gray-zone case counted wrong. Type 1 uses strict normalized exact match; type 2 uses exact match or multiple-choice letter match. For types 3 and 4 the *primary* measure is verdict plus location: did the model find the contradiction and cite the right place? The category label is a *secondary*, finer-grained measure, since the four violation categories are not perfectly mutually exclusive (e.g., a numeric change inside a cross-referenced table). A cited location is checked by distinctive-token overlap against the gold location: overlap ≥ 0.45 counts as correct, < 0.25 as wrong, and the gray zone in between is counted wrong under the conservative scoring, with no human or LLM judgment involved. As a precision guard, citations quoting more than five times the gold location’s distinctive tokens fall into the gray zone rather than auto-scoring, so the recall-style overlap check cannot be gamed with overly broad quotes. Full credit on an item requires the primary verdict-plus-location

plus the correct category. The conservative scores are fully reproducible by running `score_main.py -conservative` and `score_layered.py -conservative` in the released repository.

Separately, a deterministic lenient-threshold variant scores t3/t4 items with the location overlap bar lowered from 0.45 to 0.25 (the “not wrong” boundary); t1/t2 are unchanged, as exact match has no gray zone. It is reported only as a sensitivity analysis in Appendix A, showing how much the headline figures move under a more forgiving location rule. Distractors are correct if and only if the verdict is “consistent”. Prompts are built once per question and shared across models (verified by prompt SHA-256), so cross-model comparisons are prompt-identical. We report 95% Wilson score intervals [16] throughout.

3.5 Anti-contamination check

A with-spec vs. without-spec ablation on a 60-question sample indicates that the benchmark measures reading rather than recall: without the excerpt, accuracy is 3/60, and the three correct answers trace to two leaky type-2 questions (flagged, one since rewritten) and one multiple-choice guess (Section 5.2).

4 Dataset Statistics

The released dataset (frozen 2026-09-26) reflects a full audit pass: 41 stale type-4 excerpts were repaired against the actual mutated RTL; all 98 type-4 mutations were re-simulated and re-confirmed to differ; 10 legacy UART type-4 labels were converted to the canonical four-category scheme; and an exhaustive deduplication audit found exactly one true duplicate pair (an earlier internal report of seven pairs did not reproduce; the other six were distinct mutations).

IP block	questions
i2c	55
spi_host	55
aes	50
spi_device	50
kmac	45
hmac	35
uart	30
aon_timer	25
rv_timer	22
pattgen	18
Total	385

Table 1: SpecRead composition: 385 questions across 10 OpenTitan IP blocks (t1: 58, t2: 58, t3: 171, t4: 98), plus 41 distractor controls and 41 consistent-RTL controls.

5 Experiments

5.1 Main evaluation

We evaluated Ministral-3B (`ministral-3b-latest`, the Mistral API alias as served on 2026-09-25/26; the API exposes no static version string, so we log prompts, timestamps, and the alias for reproducibility; temperature 0; one question per call) on all 385 questions plus 41 distractors plus 41 t4 controls (467 calls, 0 errors). Table 2 reports results. The calls ran in four batches (UTC): 2026-09-25 21:xx, t1 (58), 43 unrepaired t2, t3 (171), and distractors (41); 2026-09-26 01:xx, the 15 repaired t2 items whose prompts gained the previously omitted option lists; 2026-09-26 02:xx, all 98 t4 items and 41 t4 controls with the v2.1 prompt template. A post-freeze audit then found the 09-25 t3 calls used a prompt template differing by 36 characters from the published template (a verdict-commit wording tweak; the ablation with-spec t3 records already used the published template), so all 171 t3 items were re-run on 2026-09-26 with the published template, replacing the stale records in place. Per-record timestamps and prompt SHA-256 hashes are logged in the release, making the mixed timeline auditable; after the re-run, all records are prompt-identical with the published templates.

Type	<i>n</i>	correct	acc.	95% CI	category acc.
t1 (exact retrieval)	58	32	55.2%	[0.425, 0.673]	—
t2 (cross-section reasoning)	58	30	51.7%	[0.392, 0.641]	—
t3 (spec contradiction)	171	82/171	48.0%	[0.406, 0.554]	49.7%
t4 (spec-RTL consistency)	98	62/98	63.3%	[0.534, 0.721]	43.9%
Overall (full credit)	385	128	33.2%	[0.287, 0.381]	
Macro avg. (mean of 4 types)			39.0%		
Distractor false-positive rate	41	21 FP	51.2%	[0.365, 0.657]	
t4 control false-positive rate	41	41/41	100.0%	[0.914, 1.000]	
Model-free baselines (deterministic): always-violation verdict balanced acc. t3 50.0%, t4 50.0%; random category 25.0%; t3 diff-against-upstream location 35.2% (57/162; 95% CI [0.283, 0.428]).					

Table 2: Main results, Ministral-3B (dataset v2.1), conservative scoring: gray-zone cases counted wrong. For t3/t4 the primary headline is verdict-plus-location; the category column is secondary (the four labels are not mutually exclusive). “Overall (full credit)” requires verdict, location, and category on t3/t4. Baselines are computed by script with no model calls (see text).

Retrieval is the strongest type at 55.2%, and cross-section reasoning reaches 51.7% after the multiple-choice repair. On the two contradiction-focused types, which have no generation-benchmark analogue, the primary verdict-plus-location measure gives 48.0% for spec contradiction (t3, 82/171) and 63.3% for spec-RTL consistency (t4, 62/98). Full credit (verdict, location, and category) is lower at 24.6% and 24.5%, reflecting the category-classification gap discussed below. For context, the deterministic baselines are 50.0% verdict balanced accuracy for an always-violation predictor (t3 and t4), 25.0% expected category accuracy for random labels, and 35.2% t3 location accuracy for a diff-against-upstream string comparison (57/162; 95% CI [0.283, 0.428]), which bounds how much of the location sub-task is solvable by memorized-source comparison alone.

The 51.2% distractor false-positive rate is a finding in its own right. The model is easily misled by suspicious-looking but self-consistent passages; it over-flags rather than under-flags. On the t3 verdict-level decision (171 mutated items, gold: violation; 41 distractors, gold: consistent), the

model flags a violation on 171/171 mutations and 21/41 distractors: TPR 100.0%, TNR 48.8%, balanced accuracy 74.4%. This is the one non-degenerate verdict signal in the paper: unlike t4, the model does produce the compliant verdict on self-consistent spec passages about half the time.

The t4 consistent-RTL controls sharpen this: on 41 unmutated RTL excerpts where the correct answer is `NO_CONTRADICTION`, the model flagged a violation every single time (41/41 false positives), never once producing the compliant verdict even though the prompt explicitly offered it. Combining the 98 mutations (gold: violation) with the 41 controls (gold: compliant) into one violation-vs.-compliant decision task, the model predicts “violation” on 97/98 mutations and 41/41 controls: balanced accuracy 49.5%, precision 70.3%, recall 99.0%, F1 82.2%, MCC -0.06. The near-zero MCC confirms the decision carries almost no signal; for this small model it is effectively degenerate toward “violation.”

Prompt-sensitivity check. The 100% false-positive rate on t4 controls could in principle be an artifact of the prompt’s verdict-commit instruction (“You must commit to a verdict”). To test this, we re-ran all 41 t4 controls with a variant prompt whose instruction instead begins “The RTL excerpt may be fully compliant with the specification” (everything else byte-identical; same model, temperature 0, deterministic item order). The model still flagged a violation on 41/41 controls (100.0%, 95% Wilson CI 91.4–100.0). The over-flagging bias is robust to this prompt wording; it is not explained by the verdict-commit instruction alone. (Raw responses and prompt hashes in `build/eval/responses_controls_mistral_promptvar.jsonl`.)

A 30-question pilot on OpenTitan UART preceded the full build as a difficulty check, run on `gemini-flash-lite-latest` (16/30) and `ministral-3b-latest` (14/30). Both pilot models scored below the 60% bar that would have triggered hardening, and missed most planted contradictions rather than over-flagging, which motivated the full build. (Pilot records are in `pilot/model_testing/` in the repository.) The pilot under-flagging contrasts with the main model’s over-flagging on distractors and t4 controls, a difference we attribute to the full build’s harder distractor set and the t4 control format rather than to a model change. Evaluation of larger models on the identical 467 prompts is left to a future revision; all numbers in this paper are for Ministral-3B, and we do not claim that the difficulty profile transfers to frontier models.

5.2 Contamination ablation

Table 3 compares with-spec and without-spec accuracy on the 60-question sample. The sample is one item per IP for t1/t2 and two per IP for t3/t4, taking the lowest-numbered IDs per IP: deterministic and stratified by IP, not a uniform random draw. The 60-question sample was scored with manual adjudication of gray-zone cases (not the conservative main scoring); the full-set column below uses the conservative main scores. The with-spec sample numbers exceed the conservative full-set scores on t1 (60.0% vs. 55.2%), t3 (45.0% vs. 48.0% verdict-plus-location), and overall (36.7% vs. 33.2%), and run below on t2 (40.0% vs. 51.7%) and t4 (15.0% vs. 63.3%). With $n=10-20$ per type, these differences are within small-sample noise: the 95% Wilson intervals overlap substantially (t1 sample [0.313, 0.832] vs. full set [0.425, 0.673]; t3 sample [0.258, 0.658] vs. [0.406, 0.554]). The ablation’s purpose, detecting contamination rather than estimating absolute difficulty, is unaffected: the without-spec column is what carries the claim.

The contamination conclusion from this ablation is restricted to t1 and t2: without the excerpt, t1 accuracy drops to 0.0% and t2 to 30.0%, showing these questions require reading the provided text rather than recalling training data. The small t2 delta is expected: 3 of the 10 sampled t2 questions are answerable or guessable without the excerpt, which is exactly what the ablation is designed to detect. For t3 and t4, the without-spec condition is uninformative by construction: the without-

Type	with spec	without spec	Δ
t1 ($n=10$)	6/10 = 60.0%	0/10 = 0.0%	+60.0pp
t2 ($n=10$)	4/10 = 40.0%	3/10 = 30.0%	+10.0pp
t3 ($n=20$)	9/20 = 45.0%	0/20 = 0.0%	+45.0pp
t4 ($n=20$)	3/20 = 15.0%	0/20 = 0.0%	+15.0pp
Overall	22/60 = 36.7%	3/60 = 5.0%	+31.7pp

Table 3: Contamination ablation, Ministral-3B (dataset v2.1, fixed prompts; sample scored with manual adjudication of gray-zone cases). 95% Wilson intervals for the overall row: with spec [0.256, 0.493], without spec [0.017, 0.137]. The without-spec prompts contain the question only, no excerpt and no RTL.

spec prompt contains the question only (no excerpt and, for t4, no RTL), so locating a planted contradiction or judging spec-RTL consistency is impossible without the material the question is about. We draw no contamination conclusion for t3/t4 from this ablation. Note further that even for t3, the ablation does not rule out that memorized knowledge of the *unmutated* OpenTitan text helps a model notice a mutation; contamination resistance for the mutated types rests on regenerability (fresh mutations on demand) rather than on this ablation alone.

5.3 Rule-table prompting (negative result)

A two-step prompting intervention first extracts the excerpt into a numbered rule table and then answers using only the table. On the same 60-question sample it lowers accuracy on every type except t2 (tied): t1 -0.300 (6/10 to 3/10), t2 0.000 (4/10 to 4/10), t3 -0.100 (9/20 to 7/20), t4 -0.150 (3/20 to 0/20), overall -0.133 (22/60 to 14/60). A McNemar exact test on the 60 paired outcomes gives $p = 0.039$ (10 discordant pairs favoring the standard prompt, 2 favoring rule-table), so the degradation is statistically significant at $\alpha = 0.05$. Step-1 extraction is faithful (spot-checked tables preserve the mutated numbers); the loss occurs in step 2 through recombination failures, hedging instead of committing to a verdict, and paraphrase drift penalized by strict scoring. For a 3B model, structured extraction does not help specification reading; it hurts.

6 Failure Analysis

Layered scoring separates *verdict/location* (did the model find the contradiction?) from *category* (did it label the violation type correctly?). On t3, the model cites the correct location on 135/171 items (78.9%) but the correct category on only 85/171 (49.7%); both correct on 69/171 (40.4%). On t4, using a mutation-aware location check (the cited code must contain the mutated snippet’s distinctive tokens), location accuracy is 80/98 (81.6%), category 43/98 (43.9%), both 36/98 (36.7%). The layered location bar is the lenient “not wrong” boundary (overlap ≥ 0.25), so the “both” figures exceed the conservative strict scores (42/171, 24/98), which count every gray-zone case wrong: the gap between “both” and strict is exactly the gray zone. The pattern is the same on both types: the model usually finds the discrepancy but misclassifies it. We state this as a hypothesis, not a firm conclusion: the fine-grained violation taxonomy *may* be the hard part for this model, as opposed to spotting that something is wrong. The hypothesis is confounded with label ambiguity: the four category labels have not been independently validated, and for RTL mutations `NUMERIC_MISMATCH` and `RULE_INVERSION` often overlap (e.g., a numeric change that inverts a comparison), so part

of the “misclassification” may reflect genuinely ambiguous gold labels rather than model failure. Category confusion matrices (Tables 4, 5) show the dominant errors: on t3, the model over-predicts `CROSSREF_CONFLICT`, mislabeling 19 `RULE_INVERSION` and 18 `MISSING_CONDITION` items as cross-reference conflicts. On t4 the dominant error is different: 26 `NUMERIC_MISMATCH` items mislabeled as `RULE_INVERSION` (vs. only 3 such errors on t3).

gold \ pred	CROSSREF	MISSING	NUMERIC	RULE_INV
CROSSREF_CONFLICT	24	2	9	4
MISSING_CONDITION	18	2	3	14
NUMERIC_MISMATCH	8	0	39	3
RULE_INVERSION	19	2	4	20

Table 4: t3 category confusion matrix, Ministral-3B (rows: gold, columns: predicted).

gold \ pred	CROSSREF	MISSING	NUMERIC	RULE_INV
CROSSREF_CONFLICT	1	0	3	1
MISSING_CONDITION	3	8	0	5
NUMERIC_MISMATCH	6	6	12	26
RULE_INVERSION	2	2	0	22

Table 5: t4 category confusion matrix, Ministral-3B (rows: gold, columns: predicted).¹

We observe the following characteristic failure modes for Ministral-3B.

- **Right location, wrong category (t3/t4).** The dominant mode on both types, confirmed by the layered scores above: the model finds the mutated line or conflicting statements but misclassifies them (e.g., cites the correct reset-value mutation yet labels it `RULE_INVERSION` instead of `NUMERIC_MISMATCH`).
- **Mutation spotted, conflict not identified (t3).** The model quotes the mutated sentence but pairs it with an unrelated statement instead of the one it actually conflicts with.
- **Hedging instead of committing (t3).** The model describes conditions under which a contradiction would exist rather than giving a verdict; this is worse under the rule-table prompt.
- **Wrong contradiction (t4).** The model finds a discrepancy other than the planted one (e.g., flags a word-size difference while the gold is a FIFO-ordering rule).
- **Over-flagging (distractors and t4 controls).** 21/41 false positives on self-consistent passages, and 41/41 on consistent RTL.

Two concrete cases illustrate the failure modes. On `kmac-t3-019` (gold: `CROSSREF_CONFLICT` for a mutated `ERR_CODE` link), the model correctly labeled the category but discussed the `kmac_done` interrupt versus `STATUS.squeeze` polling, a real tension in the excerpt but not the planted mutation: it found *a* conflict, not *the* conflict. On `aes-t3-001` (gold: `NUMERIC_MISMATCH`, 9/11/12 rounds

¹Gold `NUMERIC_MISMATCH` totals 51 items: one (`spi_host-t4-005`) is omitted from this table because the model response contained no parseable category label, so the displayed predictions sum to 50.

versus 12/14/16 cycles), the model quoted the mutated “9/11/12 rounds” figure but paired it with an unrelated throughput sentence instead of the conflicting cycle counts, so the contradiction was never isolated. Both cases show the model engaging with the right region of the spec while missing the precise planted inconsistency.

A natural next step is to test whether SpecRead scores predict downstream generation failures: do models that misread specifications here also commit more specification-misreading errors on VerilogEval-style generation tasks? The failure modes above (right region, wrong precise conflict) suggest the errors would transfer, but a quantitative correlation study is left to future work. By mutation operator on t3, numeric edits are the easiest to catch and dropped conditions the hardest (Table 6): the model notices altered numbers but never gets a `deleted_condition` item fully right (0/37 strict), the subtlest operator. A location-only re-analysis of the 37 `deleted_condition` items (counting a response as finding the conflict when its cited location overlaps the gold location above the automatic-wrong threshold, ignoring category) rises from 0/37 strict to 28/37 (75.7%), confirming that even on the hardest operator the model usually sees the conflict but cannot name its type. Strict exact-match scoring penalizes paraphrase-correct t1/t2 answers, but a lenient variant

Operator	correct	<i>n</i>	acc.
<code>changed_number</code>	21	53	39.6
<code>crossref_conflict</code>	9	30	30.0
<code>rule_inversion</code>	12	51	23.5
<code>deleted_condition</code>	0	37	0.0

Table 6: Type-3 accuracy by mutation operator, Ministral-3B (conservative scoring).

(the pipeline’s `score_exact_lenient`, which strips code formatting) flips only 4 of the 94 non-multiple-choice t1/t2 judgments on the full v2.1 set, so the headline numbers stand.

7 Limitations

(1) Only one small open model (Ministral-3B) is evaluated; conclusions about difficulty may not transfer to larger or frontier models, which we did not evaluate (resource constraint). (2) All sources come from a single project (OpenTitan); cross-project generality is untested. (3) Specifications are English prose plus register tables; other documentation styles (e.g., assertions as specification) are not covered. (4) Type-4 items use small, simulation-observable mutations; deep microarchitectural changes are out of scope, and a simulation difference from the original RTL is a proxy for, not a proof of, a specification violation. (5) Strict exact-match scoring conflates retrieval with formatting discipline for small models. (6) The main results use conservative scoring (gray-zone cases counted wrong). A deterministic lenient-threshold variant (location overlap ≥ 0.25) appears only in the appendix sensitivity analysis; no human annotator reviewed the 98 t4 gold labels, and the category labels have not been independently validated (see Failure Analysis). (7) The t4 control false-positive rate (41/41) rests on 41 items; the controls were fairness-audited but a larger control set would tighten the interval. Combining the 98 mutations with the 41 controls, the violation-vs.-compliant decision has balanced accuracy 49.5% and MCC -0.06 (see Results): for this small model the decision is effectively degenerate. (8) Regenerability via mutation injection applies to type-3 items; type-4 items require hand-built testbenches and simulation filtering, so they are not regenerable by the same scripted pipeline. (9) No human baseline is reported; the benchmark measures model behavior, not human difficulty.

8 Conclusion

SpecRead separates understanding a hardware specification from producing hardware artifacts. As a benchmark release with an initial single-model characterization, it shows that on a small open model there is a clear gap: 33.2% overall (128/385; macro average 39.0%), 48.0–63.3% verdict-plus-location on the two contradiction-focused types that have no generation-benchmark analogue, a 51.2% false-positive rate on self-consistent passages, and a 100% false-positive rate on consistent-RTL controls. Layered scoring shows the model usually locates contradictions (78.9–81.6%) but scores lower on their category (49.7–43.9%), suggesting as a hypothesis that the fine-grained violation taxonomy may be the hard part, subject to the label-ambiguity caveat above. We make no claim that this difficulty profile transfers to larger or frontier models. The benchmark is automatically scorable under conservative scoring, regenerable for type-3 items by mutation injection, simulation-grounded, and built entirely from public sources. The dataset, simulation harnesses, and scoring pipeline are publicly released at <https://github.com/EDGAbab/specread> so the community can measure, and improve, models’ ability to read the documents hardware is built from.

Reproducibility

Dataset v2.1 frozen 2026-09-26: 385 questions, 41 distractors, 41 t4 controls; sources pinned to OpenTitan commit 80b5b3213453156fe62683962664d1afd9695ec4 (Apache 2.0); all 98 type-4 mutations re-simulated under Icarus Verilog and confirmed to differ; temperature 0; prompt-identical cross-model calls (verified by prompt SHA-256). Dataset versions: v1 (pilot, 30 questions), v2 (full build, 385 questions), v2.1 (t2 multiple-choice repair, t4 prompt fix, 41 t4 controls added). Difficulty labels (easy/hard) are recorded per item in the release. Code and data: publicly released at <https://github.com/EDGAbab/specread>.

A Lenient-threshold scoring: sensitivity analysis

The main results count every gray-zone case wrong (location overlap below 0.45). As a deterministic sensitivity analysis, Table 7 reports the scores when the location overlap bar is lowered to 0.25 (the “not wrong” boundary used in the layered analysis); t1/t2 are unchanged, as exact match has no gray zone. This variant is fully scripted with no human or model judgment, and shows how much the headline figures move under a more forgiving location rule.

Type	<i>n</i>	correct	acc.	95% CI
t1 (exact retrieval)	58	32	55.2%	[0.425, 0.673]
t2 (cross-section reasoning)	58	30	51.7%	[0.392, 0.641]
t3 (spec contradiction)	171	69/171	40.4%	[0.333, 0.478]
t4 (spec-RTL consistency)	98	36/98	36.7%	[0.279, 0.466]
Overall	385	167/385	43.4%	[0.385, 0.484]
Macro avg. (mean of 4 types)			46.0%	

Table 7: Sensitivity analysis: deterministic lenient-threshold scores (t3/t4 location overlap ≥ 0.25). The conservative Table 2 is the main result.

The lenient variant moves t3 from 24.6% to 40.4%, t4 from 24.5% to 36.7%, and the overall figure from 33.2% to 43.4%. The qualitative pattern is unchanged: retrieval and cross-section reasoning

stay well above the contradiction-focused types, and the layered location-vs.-category gap persists. The conservative main results are therefore a lower bound on what a more forgiving location rule would credit, and the ranking of types by difficulty is the same under either scoring.

References

- [1] M. Liu, N. Pinckney, B. Khailany, and H. Ren. VerilogEval: Evaluating large language models for Verilog code generation. In *ICCAD*, 2023. arXiv:2309.07544.
- [2] N. Pinckney, C. Batten, M. Liu, H. Ren, and B. Khailany. Revisiting VerilogEval: Newer LLMs, in-context learning, and specification-to-RTL tasks. arXiv:2408.11053, 2024.
- [3] Y. Lu, S. Liu, Q. Zhang, and Z. Xie. RTLLM: An open-source benchmark for design RTL generation with large language model. In *ASP-DAC*, 2024. arXiv:2308.05345.
- [4] W. Fang, M. Li, M. Li, Z. Yan, S. Liu, H. Zhang, and Z. Xie. AssertLLM: Generating and evaluating hardware verification assertions from design specifications via multi-LLMs. In *ASP-DAC*, 2025. arXiv:2402.00386.
- [5] Y. Wu, W. Fang, J. Wang, W. Li, Z. Guo, and Z. Xie. AssertLLM2: A comprehensive LLM benchmark for assertion generation from design specifications. arXiv:2605.27472, 2026.
- [6] R. Qiu, G. L. Zhang, R. Drechsler, U. Schlichtmann, and B. Li. AutoBench: Automatic test-bench generation and evaluation using LLMs for HDL design. arXiv:2407.03891, 2024.
- [7] J. Wang, S. Liu, W. Fang, Y. Wu, Y. Zhu, and Z. Xie. RTL-BenchLS: A large-scale benchmark for RTL reasoning and generation with large language models. arXiv:2606.08976, 2026.
- [8] N. Pinckney, C. Deng, C.-T. Ho, Y.-D. Tsai, M. Liu, W. Zhou, B. Khailany, and H. Ren. Comprehensive Verilog design problems: A next-generation benchmark dataset for evaluating large language models and agents on RTL design and verification. arXiv:2506.14074, 2025.
- [9] Z. Zhao, B. Li, Y. Li, Z. Yan, and U. Schlichtmann. VClare: Resolving imperfect specifications in LLM-based Verilog generation. arXiv:2607.24854, 2026.
- [10] J. Zhang, C. Liu, L. Cheng, X. Li, and H. Li. Understanding and mitigating errors of LLM-generated RTL code. arXiv:2508.05266, 2025.
- [11] M. Liu et al. ChipNeMo: Domain-adapted LLMs for chip design. arXiv:2311.00176, 2023.
- [12] D. Kiela et al. Dynabench: Rethinking benchmarking in NLP. In *NAACL*, 2021.
- [13] C. White et al. LiveBench: A challenging, contamination-limited LLM benchmark. arXiv:2406.19314, 2024.
- [14] lowRISC and the OpenTitan contributors. OpenTitan. <https://github.com/lowRISC/opentitan>.
- [15] S. Williams. Icarus Verilog. <https://github.com/steveicarus/iverilog>.
- [16] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *JASA*, 22(158):209–212, 1927.

- [17] W. Yin, D. Radev, and C. Xiong. DocNLI: A large-scale dataset for document-level natural language inference. In *Findings of ACL*, 2021. arXiv:2106.09449.
- [18] Y. Koreeda and C. Manning. ContractNLI: A dataset for document-level natural language inference for contracts. arXiv:2110.01799, 2021.
- [19] P. Laban, T. Schnabel, P. N. Bennett, and M. A. Hearst. SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. *TACL*, 2022. arXiv:2111.09525, 2021.
- [20] H. Liu, L. Cui, J. Liu, and Y. Zhang. Natural language inference in context: investigating contextual reasoning over long texts. In *Proc. AAAI*, 2021. arXiv:2011.04864.
- [21] L. Shi, M. Kazda, B. Sears, N. Shropshire, and R. Puri. Ask-EDA: A design assistant empowered by LLM, hybrid RAG and abbreviation de-hallucination. In *Proc. First IEEE Int. Workshop on LLM-Aided Design (LAD)*, 2024. arXiv:2406.06575.
- [22] Y. Pu, Z. He, T. Qiu, H. Wu, and B. Yu. Customized retrieval augmented generation and benchmarking for EDA tool documentation QA. arXiv:2407.15353, 2024.
- [23] B.-Y. Wu, U. Sharma, S. R. D. Kankipati, A. Yadav, B. K. George, S. R. Guntupalli, A. Rovinski, and V. A. Chhabria. EDA Corpus: A large language model dataset for enhanced interaction with OpenROAD. arXiv:2405.06676, 2024.